COMPLETE PROJECT DOCUMENTATION PACKAGE

Everything You Need to Start the Internship Project

WHAT YOU HAVE (5 Documents)

1. complete_project_walkthrough.md (MOST IMPORTANT)

Purpose: The complete, relentless walkthrough of ALL 24 decisions and their implementations.

Contains:

- Q4-Q24: Every critical decision with recommended answer
- Full technical justification for each recommendation
- Implementation code (copy-paste ready)
- Month-by-month breakdown
- Risk assessment and mitigation
- Final deliverables checklist

When to use:

- **READ FIRST:** Before starting Month 1 Week 1
- **REFERENCE:** When stuck on architecture decisions
- **CHECK:** Against monthly deliverables to stay on track

Key sections:

- Phase 0: Bootstrap annotation system
- Phase 1 (Month 1): Data preparation
- Phase 2 (Month 2): ML & core API
- Phase 3 (Month 3): Integration & metrics
- Phase 4 (Month 4): Production systems

Time to read: ~2 hours (skim) or ~6 hours (deep study)

---

2. optimal_system_prompt.md (USE FOR CLAUDE/CODEX)

Purpose: Comprehensive system prompt for AI code generation.

Contains:

- Project overview & timeline
- Tech stack specifications
- 14 locked architecture decisions
- File organization structure
- Coding standards (type hints, docstrings, logging, testing)
- How to request code generation (clear formats)
- Performance gates & quality criteria
- Common gotchas to avoid
- Deliverable checklists by month

When to use:

- **USE AS BASE:** When starting a multi-file coding task with Claude/Codex
- **CUSTOMIZE:** For specific month/phase (change Month/Week/Phase info)
- **REFERENCE:** For coding style questions ("should I use logging?", "where should this function go?")

Time to read: ~30 minutes (skim) or ~90 minutes (detailed study)

---

3. quick_paste_prompt.txt (COPY-PASTE DAILY)

Purpose: Condensed system prompt for quick daily setup.

Contains:

- Project summary (1 paragraph)
- Tech stack (5 lines)
- 13 locked decisions (digest format)
- File structure (tree view)
- Coding rules (quick checklist)
- Request format template
- Monthly deliverables (bullet summary)
- Common gotchas (don't/do pairs)

When to use:

- **PASTE AT SESSION START:** Every time you open Claude/Codex
- **CUSTOMIZE:** Add current month/week/status
- **FOLLOW:** The request format when asking for code

Copy-paste time: <1 minute

---

4. session_template.md (USE EVERY CODING SESSION)

Purpose: Template structure for each Claude/Codex session.

Contains:

- Session setup checklist
- 5 example sessions (Month 1 Week 1 → Month 4 Week 14)
- Common questions during session
- End-of-session checklist
- Quick reference table
- Full session example (copy this exactly)

When to use:

- **START EVERY SESSION:** Use the template structure
- **FILL IN:** Current status, today's goal, time available
- **REFERENCE:** Example sessions for how to phrase requests
- **END CHECKLIST:** Before ending coding session

Time per session: ~5 minutes to set up

---

5. non_ml_infrastructure_checklist.md (REFERENCE FOR BACKEND)

Purpose: Non-ML code templates and implementation guidance.

Contains:

- Database schema (PostgreSQL)
- File processing pipeline (PDF, CSV, Excel, OCR)
- Label Studio integration
- Data cleaning utilities
- FastAPI application structure
- Database ORM + queries
- Orchestration logic
- KPI calculation
- Dashboard frontend
- Monitoring & alerting

When to use:

- **CONSULT:** When building backend/infrastructure components
- **COPY PATTERNS:** For database models, API routes, utilities
- **ADAPT:** Code snippets to your specific fields

Time to reference: ~1 minute per code snippet

HOW TO USE THESE DOCUMENTS (TIMELINE)

BEFORE MONTH 1 STARTS

```
1. Read complete_project_walkthrough.md (full version)
   - Time: 6 hours
   - Goal: Understand entire project, all decisions are locked

2. Read optimal_system_prompt.md
   - Time: 1.5 hours
   - Goal: Know coding standards, architecture, quality gates

3. Skim session_template.md examples
   - Time: 30 minutes
   - Goal: See how to structure requests to Claude
```

MONTH 1 WEEK 1 (Starting Project)

```
1. Copy quick_paste_prompt.txt into a new Claude session
2. Fill in Month/Week/Status
3. Add "Month 1 Week 1 - Starting" to session details
4. Follow the "current status" template for what's done/in progress
5. Use the request format when asking Claude for code
```

EVERY CODING SESSION (Weeks 1-16)

```
1. Paste quick_paste_prompt.txt at top
2. Add session_template.md structure below it
3. Fill in current status (what's done, what's in progress, what's blocked)
4. Fill in today's goal and time available
5. Paste code context if file is > 100 lines
6. Follow the request format template
7. At end of session: run end-of-session checklist
8. Commit to GitHub with clear message
9. Update status for next session
```

WHEN STUCK ON DECISIONS

```
1. Refer to complete_project_walkthrough.md Q[number] section
2. Read the recommendation + justification
3. Check if dependencies are met (Q's listed in "Dependency" section)
4. If still confused: ask Claude with full context
```

WHEN WRITING CODE

```
1. Check optimal_system_prompt.md "Coding Rules" section
2. Follow file structure from optimal_system_prompt.md
3. Use patterns from non_ml_infrastructure_checklist.md
4. Follow Python + type hints + docstrings + logging
5. Write tests as you code
6. Run quality checks:
   - mypy app/ (type checking)
   - flake8 app/ (PEP 8)
   - pytest app/ (tests)
```

END OF MONTH REVIEW

```
1. Check Month X "Summary" section in complete_project_walkthrough.md
2. Compare your deliverables to checklist
3. If behind: adjust Month+1 schedule
4. If ahead: use extra time for optimization/testing
5. Commit all code with "Month X Complete" message
```

QUICK REFERENCE: FILE LOCATIONS

Question	Document	Section
"What should I build in Month 2?"	complete_project_walkthrough.md	PHASE 2: MONTH 2
"How should I structure my code?"	optimal_system_prompt.md	File Organization
"What are my coding rules?"	optimal_system_prompt.md	Coding Standards
"How do I request code from Claude?"	session_template.md	REQUEST FORMAT
"What's the quickest prompt to use?"	quick_paste_prompt.txt	Copy entire file

Question	Document	Section
"How do I build the database?"	non_ml_infrastructure_checklist.md	Database Schema
"Show me API examples"	non_ml_infrastructure_checklist.md	FastAPI section
"What's Q15 about?"	complete_project_walkthrough.md	Q15: Confidence Thresholding

---

RECOMMENDED READING ORDER (First Time Only)

Duration: ~10 hours over 2-3 days

Day 1 (3 hours):

```
1. Read this file (summary) - 15 min
2. Skim complete_project_walkthrough.md intro + architecture - 1 hour
3. Read optimal_system_prompt.md once - 1 hour
4. Skim session_template.md examples - 45 min
```

Day 2 (4 hours):

```
1. Read complete_project_walkthrough.md PHASE 1 (Month 1) - 1 hour
2. Read complete_project_walkthrough.md PHASE 2 (Month 2) - 1.5 hours
3. Read non_ml_infrastructure_checklist.md (skim database + file processing) -
1.5 hours
```

Day 3 (3 hours):

```
1. Read complete_project_walkthrough.md PHASE 3 (Month 3) - 1 hour
2. Read complete_project_walkthrough.md PHASE 4 (Month 4) - 1 hour
3. Re-read quick_paste_prompt.txt (you'll use this daily) - 30 min
```

Total: ~10 hours investment → saves 40+ hours of decision-making during project

---

HOW TO CUSTOMIZE FOR YOUR SPECIFIC DOMAIN

If your host organization has **different document types or fields**, you'll customize:

1. **Extraction Schema (Q7)**
    - Instead of: {INVOICE_ID, INVOICE_DATE, VENDOR_NAME, TOTAL_AMOUNT}
    - Use: {YOUR_FIELD_1, YOUR_FIELD_2, ...}

- Update everywhere in code

2. **Business Rules (Q16 - Anomaly Detection)**
   - Instead of: "Amount > $1M is suspicious"
   - Use: Your domain-specific rules
   - Update `BusinessRuleAnomalyDetector` class
3. **KPIs (Q17)**
   - Instead of: Documents processed, extraction rate, accuracy
   - Add: Domain-specific metrics (e.g., "compliance rate", "processing cost")
   - Update `KPIEngine` in Month 3
4. **File Types (Month 1)**
   - Instead of: {PDF, CSV, Excel, scans}
   - Add: Your actual document types
   - Update `FileHandler.ALLOWED_TYPES`

Rest of project stays the same.

GITHUB SETUP (Recommended)

Initialize before Month 1 Week 1:

```bash
# Create repo
git init
git add .
git commit -m "Month 1 Week 1: Initial commit - project structure"

# Make commits like this throughout:
git commit -m "Month 1 Week 2: Label Studio setup, annotation guidelines"
git commit -m "Month 1 Week 3: Data collection (50/200 documents)"
git commit -m "Month 1 Week 4: Annotation complete, IIA=0.82"

git commit -m "Month 2 Week 5: Random Forest model (88% accuracy)"
git commit -m "Month 2 Week 6: spaCy NER model (87% F1)"
git commit -m "Month 2 Week 7: FastAPI endpoints, extraction pipeline"
git commit -m "Month 2 Week 8: Testing complete, 86% end-to-end F1"

# ... etc for Months 3 & 4
```

This creates a clear history of progress.

SUPPORT MATRIX

Help Needed	Use This	Expected Time
"What should I code in Week X?"	complete_project_walkthrough.md	5-10 min to find answer
"How do I phrase requests to Claude?"	session_template.md	2-3 min to copy example
"What coding rules apply?"	optimal_system_prompt.md + quick_paste_prompt.txt	1-2 min reference
"Show me how to implement X"	non_ml_infrastructure_checklist.md	5-10 min to find pattern
"Why was this architecture decided?"	complete_project_walkthrough.md (Justification section of Q#)	5 min to read reasoning
"I'm blocked on a decision"	complete_project_walkthrough.md (Dependency notes)	10-15 min to resolve
"Full session setup"	quick_paste_prompt.txt + session_template.md	<5 min copy-paste

---

WHAT SUCCESS LOOKS LIKE (End of Month 4)

All these documents will have been used to build:

✅ Data Engineering (Month 1)

- 50-200 span-level annotated documents
- Label Studio instance with export working
- Database with clean schema

✅ ML Engineering (Month 2)

- Random Forest confidence model (88%+ accuracy)
- spaCy NER model (87%+ F1)
- BERT-based document classifier
- Working extraction pipeline

✅ System Integration (Month 3)

- Confidence thresholding (per-field tuning)

- Anomaly detection (rules + statistical)
- KPI engine (operational + quality + business metrics)
- 100 documents through full pipeline (94% accuracy)

✅ **Production (Month 4)**

- FastAPI REST + async endpoints
- Streamlit dashboard with live KPIs
- Human feedback collection UI
- Monthly retraining pipeline
- Model versioning & A/B testing
- Drift detection & monitoring
- Complete documentation

All code:

- Type-hinted
- Tested (80%+ coverage)
- Logged at appropriate levels
- In GitHub with clear commits
- Follows PEP 8
- Production-ready

FINAL CHECKLIST (Before You Start Coding)

- [ ] Read this summary document (you're here ✓)
- [ ] Download/save all 5 documents
- [ ] Read complete_project_walkthrough.md (full, takes time)
- [ ] Skim optimal_system_prompt.md (understand standards)
- [ ] Review session_template.md examples (see how to structure)
- [ ] Copy quick_paste_prompt.txt to clipboard (paste at session start)
- [ ] Create GitHub repo for project
- [ ] Set up PostgreSQL locally (or Docker)
- [ ] Create project directory structure
- [ ] Open Claude/Codex and paste quick_paste_prompt.txt
- [ ] Say "Month 1 Week 1: Let's begin" and start with Q4

You're ready. 🚀

NEED HELP?

If you get stuck:

1. **Architecture question:** Check complete_project_walkthrough.md Q[number]
2. **Code question:** Check optimal_system_prompt.md "Coding Rules" or non_ml_infrastructure_checklist.md
3. **Session structure:** Copy session_template.md format
4. **Quick answers:** Use quick_paste_prompt.txt in Claude
5. **Still stuck:** Describe blocker clearly (see session_template.md examples)

The answer is almost always in one of these 5 documents.

Good luck. You've got everything you need. 🚀